

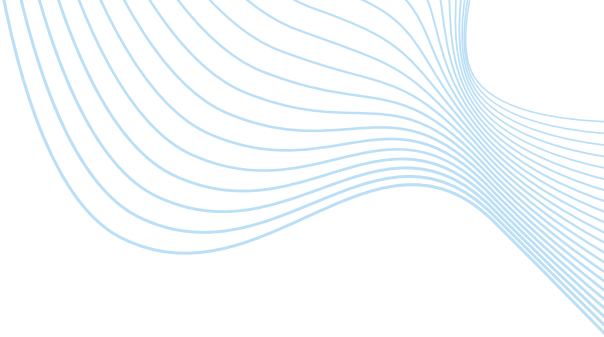


DEVELOPERSHUB CORPORATION

Web Application Security

Cybersecurity internship Report

Zainab Maqbool
May 2026



Executive Summary

In this report document, I have completed this 3-week cybersecurity task to assess a vulnerable web application, implement security fixes, and verify those improvements through penetration testing and logging. All testing was performed safely on my local machine only, and no real or public website was affected. Across all three weeks, 5 vulnerabilities were identified, 4 security controls were implemented, and 6 OWASP Top 10 categories were addressed.

The project demonstrates a complete security cycle: find → fix → verify.

Week	Application	Focus	Key Outcome
Week 1	DVWA	Vulnerability Assessment	Found XSS, SQL Injection, Weak Passwords
Week 2	OWASP NodeGoat	Security Implementation	Fixed with bcrypt, JWT, Helmet, validator
Week 3	OWASP NodeGoat	Pen Testing & Logging	Nmap scan, Winston logging, final checklist

Week 1 — Vulnerability Assessment

Application: DVWA (Damn Vulnerable Web Application)

I installed DVWA (a practice application intentionally built with security weaknesses for learning purposes) locally using XAMPP on localhost.

Vulnerability 1 — XSS (Cross-Site Scripting)

Danger level: High

Location: /dvwa/vulnerabilities/xss_r/

I tested for XSS by entering the `<script>alert('XSS Attack!');</script>` in the name input field.

A popup appeared on the screen, which confirmed that the application executed the injected JavaScript. From this, I observed that the input was not being properly sanitized before being displayed back to the user. This kind of weakness can allow an attacker to run code in a victim's browser, steal session cookies, redirect users, or perform actions on their behalf.

Vulnerability 2 — SQL Injection)

Danger level: Critical

Location: /dvwa/vulnerabilities/sqli/

To test for SQL injection, I entered the `1' OR '1'='1` in the user id field:

The application returned all user records, including admin, Gordon, Hack, Pablo, and Bob. This showed that the input was being interpreted as part of the database query instead of being treated as normal user input. From this test, I confirmed that unauthorized users could potentially access sensitive data or bypass authentication.

Vulnerability 3 — Weak Password Storage (MD5)

Danger level: High

Location: dvwa.users table (db)

I found that passwords were stored using the MD5 hashing algorithm. Since MD5 is considered outdated and insecure, I tested how easily a stored password could be recovered. I copied the admin password hash from phpMyAdmin and checked it using the online hash lookup service md5decrypt.net.

Hash: 5f4dcc3b5aa765d61d8327deb882cf99

Result: password

The hash was cracked in under two seconds. This showed that weak hashing methods make stored credentials highly vulnerable and easy to recover.

Week 1 Findings Summary

No.	Vulnerability	Danger Level	Proof	Location
1	XSS — Cross Site Scripting	HIGH	Alert popup appeared	/dvwa/vulnerabilities/xss_r/
2	SQL Injection	CRITICAL	All DB users exposed	/dvwa/vulnerabilities/sqli/
3	Weak Password Storage (MD5)	HIGH	Password cracked in 2 seconds	Database — dvwa.users table

Week 2 — Security Implementation

Application: OWASP NodeGoat (localhost:4000)

In this Week, I implement practical security improvements. I also integrated four security libraries into the NodeGoat application to address the weaknesses I had observed during Week 1.

Fix 1 — Input Validation with validator.js

Status: Completed

I installed validator.js and added it as middleware on the /login route in index.js. Before any login request reaches the database, the application now checks whether the email format is valid and whether required input is properly provided. If the input fails validation, the request is rejected immediately instead of being processed further. This helps reduce the risk of malicious input reaching the backend and addresses OWASP A03:2021 — **Injection**.

Fix 2 — Password Hashing with bcrypt

Status: Completed

I replaced the previous insecure password storage in user-dao.js with bcrypt. Every new password is now hashed with a randomly generated salt before being stored in the database. This means that even if the database is compromised, the original passwords are much harder to recover. Unlike MD5, bcrypt is intentionally slow and computationally expensive, which makes large-scale cracking attacks far less practical. This addresses OWASP A07:2021 — **Broken Authentication**.

Fix 3 — Token Authentication with JSON Web Token

Status: Completed

After a successful login, I configured the application to generate a signed JSON Web Token and store it as an `httpOnly` cookie. Because the cookie is marked as `httpOnly`, JavaScript running in the browser cannot directly access the token. This helps protect it from theft through XSS attacks. I also set the token to expire after one hour, which limits the impact if it is ever compromised.

Fix 4 — Secure HTTP Headers with Helmet.js

Status: Completed

I added `Helmet.js` to strengthen the application's default browser security protections. With this single change, the application automatically sends multiple security-related HTTP headers. These headers help protect against clickjacking, MIME-type sniffing, cross-site scripting, and accidental information disclosure.

Week 2 Findings Summary

Task	Fix Applied	Library	OWASP Category	Status
Input Validation	<code>validator.isEmpty()</code> on all form fields	<i>validator.js</i>	A03 — Injection	✓ Done
Password Hashing	<code>bcrypt.hashSync()</code> with <code>genSaltSync()</code>	<i>bcrypt</i>	A07 — Broken Auth	✓ Done
Token Authentication	<code>jwt.sign()</code> with <code>httpOnly</code> cookie, 1h expiry	<i>jsonwebtoken</i>	A07 — Broken Auth	✓ Done
Secure HTTP Headers	<code>app.use(helmet())</code> — 14+ headers set	<i>helmet.js</i>	A06 — Misconfiguration	✓ Done

Week 3 — Advanced Security and Final Reporting

Application: OWASP NodeGoat (localhost:4000)

In this Week, I verify that the security fixes from Week 2 were working correctly, identifying any remaining risks through penetration testing and adding a logging system to monitor security events in real time.

Task 1 — Penetration Testing with Nmap

Status: Completed

I used Nmap to examine which services were exposed on the local machine. This helped me understand what an attacker would be able to discover during an initial scan. Command: `nmap -sV localhost` (The `-sV` flag detects the service version running on each open port).

Port	State	Service	Risk Level
4000/tcp	open	HTTP — NodeGoat app	Expected — monitor
27017/tcp	open	MongoDB	HIGH — no auth, exposed
3306/tcp	filtered	MySQL	Low — filtered
22/tcp	closed	SSH	Low — not running

During the Nmap scan, i found that MongoDB was running on port 27017 without authentication enabled. This means the database was directly accessible from the network. In a real production environment, this could expose sensitive data so access should be limited to localhost or protected by firewall rules.

Browser-Based Attack Simulation

After the Nmap scan I manually tested common browser based attacks to confirm which vulnerabilities still existed in the original NodeGoat code and which protections were visible after the Week 2 changes.

Attack	Tool Used	Result
XSS Reflected	Browser DevTools	Alert popup confirmed
SQL Injection	Browser — manual input	Login bypass confirmed
Brute Force	Manual repeat attempts	No lockout — risk identified
Header check	Chrome Network tab	Helmet headers present

These tests showed that some security improvements were active, while additional protections such as account lockout still remained recommended.

Task 2 — Security Logging with Winston

Status: Completed

I installed Winston to record important security-related events both in the terminal and in a persistent log file named security.log. After restarting the application, the terminal displayed: `{"level":"info","message":"Application started"}`

The security.log file was automatically created in the project root and stored a permanent record of this event. I also configured login attempts, including successful and failed logins, to be logged with usernames so that suspicious patterns such as repeated brute-force attempts could be identified.

This addresses OWASP A09:2021 — **Security Logging and Monitoring Failures**.

Task 3 — Final Security Checklist

Status: Completed

No.	Security Best Practice	Status	Week
1	Validate all user inputs server-side	✓ Done	Week 2
2	Hash and salt all passwords with bcrypt	✓ Done	Week 2
3	Use token-based authentication (JWT)	✓ Done	Week 2
4	Set secure HTTP headers with Helmet.js	✓ Done	Week 2
5	Log all security events with Winston	✓ Done	Week 3
7	Use HTTPS for all data transmission	Recommended	Production
8	Account lockout after failed logins	Recommended	Production
9	Sanitize error messages shown to users	Recommended	Production
10	Keep all npm dependencies up to date	Recommended	Ongoing

Conclusion

This 3-week cybersecurity internship helped me understand the full security process of a web application identifying vulnerabilities, applying fixes, and verifying those improvements through testing and logging.

During the project, I identified key vulnerabilities, implemented security controls such as input validation, password hashing, token authentication, and secure headers, and performed penetration testing and security logging.

The main lesson I learned was that web security requires multiple layers of protection, continuous testing, and proper monitoring.